REDIS · ZERO TO HERO

Phase 2
Redis Fundamentals

A hand-note booklet to read, understand, and practice the five core Redis data structures before moving into SWE, System Design, and AI/ML use cases.

Caching Concept → **Redis Fundamentals** → SWE Use Cases →
System Design → AI/ML Use Cases → Agentic AI Use Cases → Production

START HERE

How to Use This Booklet

Read once for understanding, then use the checklists to practice hands-on.

This booklet is the "read and understand" companion to your hands-on Jupyter notebook (`phase2_redis_fundamentals.ipynb`) and the standalone project (`simple_cache_server/`). The notebook is where you run real code against a real Redis server. This booklet is what you read on your phone or print out — to remember **why** each piece exists, not just the command syntax.

> **Suggested flow:** Read one data-structure section here → open the notebook and run that section's code cells yourself → come back and tick off the practice box before moving to the next section.

Contents

01 What is Redis? & Installing it — page 3

02 String — the simplest building block — page 4

03 Hash — objects with fields — page 5

04 List — ordered queues — page 6

05 Set — unique collections — page 7

06 Sorted Set — ranked collections — page 8

07 Full Command Cheat Sheet — page 9

08 Build — Project 1: Simple Cache Server — page 10

09 Practice Checklist — page 11

10 What's Next — Phase 3 Preview — page 12

What is Redis, Really?

RE-mote DI-ctionary Server — an in-memory data structure store.

Think of Redis as a giant Python dictionary that (1) lives in **RAM**, so reads and writes take microseconds instead of milliseconds, and (2) is **shared** across every server in your system — unlike an in-process Python dict which only exists inside one running program.

Normal Database (Postgres/MySQL)	Redis
Stored on disk	Stored in RAM
Millisecond-level speed	Microsecond-level speed
Complex queries, joins	Simple, direct key-based lookups
Source of truth	Fast-access layer in front of the source of truth
Durable by default	Volatile unless you configure persistence (RDB/AOF)

Key mindset: Redis does not replace your database. It sits in front of it as a speed layer — this is the entire idea behind the Cache-Aside pattern you'll build on page 10.

Installing Redis

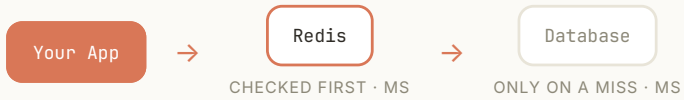
Docker (recommended)

```
docker run -d -p 6379:6379 redis:7
```

Ubuntu / WSL

```
sudo apt-get install redis-server
```

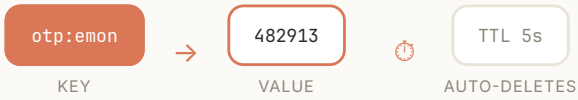
Verify it's alive: `redis-cli ping` should reply `PONG`. Python side: `pip install redis`.



the mental model behind every pattern in this booklet: Redis sits in front of, not instead of, your database

String

One key → one value. The simplest and most-used Redis type.




```
SET otp:emon 482913
EXPIRE otp:emon 5        # self-destructs in 5 seconds
TTL otp:emon           → 5  # seconds remaining
INCR api_calls:user_1  # atomic +1, race-condition safe
```

TRY IT YOURSELF

Open `redis-cli` and cache your own name for 10 seconds: `SET me "Emon" EX 10`, then run `GET me` twice — once right away, once after 11 seconds.

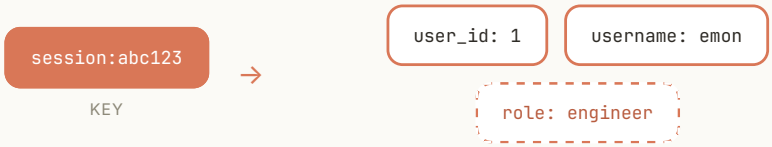
- Caching one LLM answer or API response
- Counters — views, retries, API calls
- Feature flags / config values

Command	Purpose
`SET` / `GET`	Store / read a value
`DEL`	Remove a key
`EXPIRE` / `TTL`	Set a lifespan / check seconds remaining
`SETEX` (or `SET ... EX`)	Set value + expiry in one atomic step
`INCR` / `INCRBY`	Atomic counter increment — no race conditions

> **Why INCR matters more than it looks:** if two requests hit the same counter at the same millisecond, "read → add 1 → write back" in your own Python code can silently lose an update. Redis's `INCR` is atomic — this exact idea is the foundation of rate limiting in Phase 3.

Hash

A key that holds a group of field–value pairs — a mini-dictionary inside Redis.




```
HSET session:abc123 user_id 1 username emon role engineer
HGETALL session:abc123
→ {"user_id":"1","username":"emon","role":"engineer"}
HSET session:abc123 role senior_engineer  # update ONE field only
```

TRY IT YOURSELF

Model your own profile: `HSET me:profile name Emon city Dhaka goal "AI Architect"`, then update just one field with `HSET me:profile city Remote` and confirm the rest stayed intact with `HGETALL`.

The whole point of Hash: update a single field without rewriting the entire object. Compare this to a String holding a JSON blob — that forces you to re-write everything just to change one value.

Command	Purpose
`HSET`	Set one or more fields on a hash
`HGET`	Read a single field
`HGETALL`	Read every field at once
`HDEL`	Remove a field
`HINCRBY`	Atomically increment a numeric field

- User sessions
- Object profiles (user, product)
- Shopping-cart line items

Decision rule — String vs Hash: one opaque blob you always read/write whole → **String**. An object where individual fields change independently and often → **Hash**.

List

An ordered collection — push to either end, pop from either end.




```
RPUSH job_queue send_email:user_1
RPUSH job_queue generate_report:user_2
LPOP job_queue   → send_email:user_1  # oldest job out first
```

TRY IT YOURSELF

Build a 3-item to-do queue: `RPUSH todo "wake up" "learn redis" "sleep"`, then drain it in order with `LPOP todo` three times and watch the FIFO order.

Command	Purpose
`LPUSH` / `RPUSH`	Push to the left / right end
`LPOP` / `RPOP`	Pop from the left / right end
`LRANGE`	Read a range of elements
`LLEN`	Count elements in the list

Interview-favorite detail: `RPUSH` + `LPOP` = FIFO (fair queue — first job in is first job out, what you want almost always). `RPUSH` + `RPOP` = LIFO (stack).

- Background job queues
- Message queues (Celery uses this exact idea)
- Recent-activity feeds

Set

An unordered collection of unique values. Redis enforces "no duplicates" for you.



struck-through = duplicate insert, silently ignored by Redis

```
SADD post:42:viewers emon sara emon raju
SMEMBERS post:42:viewers  → {emon, sara, raju}  # emon only once
SISMEMBER post:42:viewers sara  → 1
```

TRY IT YOURSELF

Track your own reading list with duplicates on purpose: `SADD books "Atomic Habits" "Deep Work" "Atomic Habits"`, then run `SCARD books` — confirm it says 2, not 3.

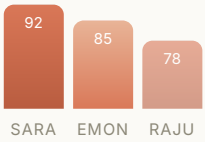
Command	Purpose
`SADD`	Add a member (ignored if already present)
`SMEMBERS`	List every member
`SISMEMBER`	Check membership — O(1) fast
`SCARD`	Count members
`SREM`	Remove a member

- Unique visitor / view tracking
- Tags on a post or product
- "Has this user already done X" checks

> Sets also support algebra — intersections and unions — so you can answer things like "users who like both Python **and** ML" in one command.

Sorted Set (ZSET)

Like a Set, but every member carries a score — Redis keeps it sorted automatically.




```
ZADD leaderboard:quiz1 85 emon 92 sara 78 raju
ZREVRANGE leaderboard:quiz1 0 -1 WITHSCORES
→ sara:92, emon:85, raju:78   # highest first, always sorted
ZINCRBY leaderboard:quiz1 5 emon   # bonus points
```

TRY IT YOURSELF

Build a 3-person leaderboard for a habit tracker: `ZADD streaks 12 emon 20 sara 7 raju`, then check who's #1 with `ZREVRANGE streaks 0 0 WITHSCORES`.

Command	Purpose
`ZADD`	Add a member with a score
`ZRANGE` / `ZREVRANGE`	Read lowest→highest / highest→lowest
`ZSCORE`	Read one member's score
`ZINCRBY`	Atomically bump a score
`ZRANK` / `ZREVRANK`	Get a member's position/rank

- Leaderboards, top-N lists
- Priority queues
- Sliding-window rate limiting (score = timestamp)

> **Spoiler for Phase 3:** a rate limiter is often just a Sorted Set where the score is the request's timestamp — old entries fall out of the window automatically.

Full Command Cheat Sheet

Everything from this phase, one page, for quick recall.

Structure	Commands	Real-world use case
String	`SET` `GET` `DEL` `SETEX` `EXPIRE` `TTL` `INCR`	LLM response cache, counters, flags
Hash	`HSET` `HGET` `HGETALL` `HDEL` `HINCRBY`	Sessions, object profiles
List	`LPUSH` `RPUSH` `LPOP` `RPOP` `LRANGE` `LLEN`	Job queues, message queues
Set	`SADD` `SMEMBERS` `SISMEMBER` `SREM` `SCARD`	Dedup, unique visitors, tags
Sorted Set	`ZADD` `ZRANGE` `ZREVRANGE` `ZSCORE` `ZINCRBY`	Leaderboards, rate limiting

The one rule that matters more than memorizing commands: ask "what *shape* is my data, and what operation needs to be fast?" — not "which structure have I heard of." That question decides 90% of good Redis design.

Quick self-test — before turning the page, try answering without looking up: which structure would you use for (a) a per-user daily API quota, (b) a live "who's online now" list, (c) a most-liked-posts-this-week feed? *(Answers: String+INCR, Set, Sorted Set.)*

If you already know Python, here's the mental shortcut

Redis structure	Closest Python equivalent	Key difference to remember
String	`variable = value`	Persists outside your process, has a TTL
Hash	`dict` / object	Update one field without reading/writing the whole thing
List	`list` / `collections.deque`	Shared across processes — a real queue, not just local memory
Set	`set`	Same uniqueness guarantee, but network-accessible
Sorted Set	`heapq` / sorted `dict`	Always sorted, O(log N) inserts, ranks built in

Why this shortcut matters for you specifically: as an AI/ML engineer, you already reach for dicts, lists, and sets constantly in Python. Redis isn't new concepts — it's the exact same mental models, made durable and shared. That's the whole reason Phase 2 goes fast once it clicks.

Project 1 — Simple Cache Server

Cache-Aside pattern: FastAPI → Redis → Fake Database.

Flow: a request arrives for a `user_id` → check Redis first → **hit**: return instantly → **miss**: query the (slow) database, store the result in Redis with a TTL, then return it. The next request for that same user, within the TTL window, is now a hit.

```python
# cache-aside, the whole idea in 6 lines
def get_user_cached(user_id):
    cached = r.get(f"user:{user_id}")
    if cached: return json.loads(cached)      # HIT
    data = slow_database_query(user_id)               # MISS
    r.set(f"user:{user_id}", json.dumps(data), ex=30)
    return data
```

Measured result

Cache miss: `~200ms`

Cache hit: `<1ms`

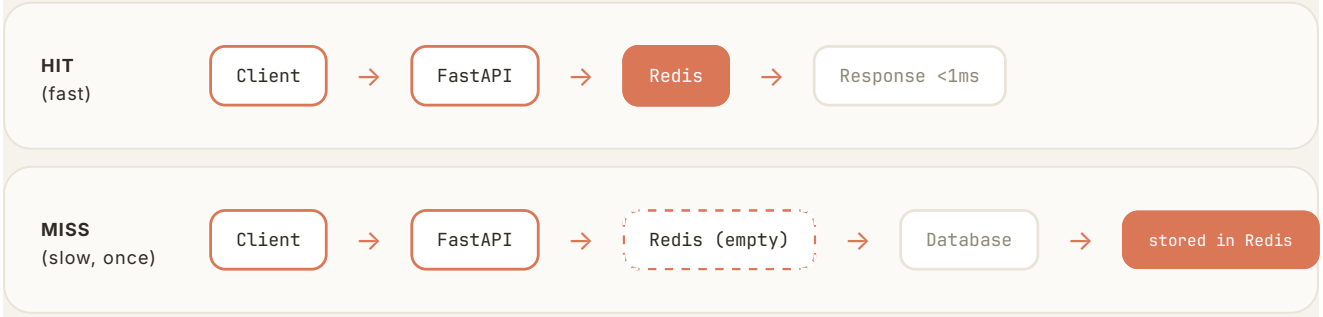
≈ 800x faster on a hit

Full runnable version

See `simple_cache_server/main.py` — run with `uvicorn main:app --reload`

Cache invalidation — deleting `user:{id}` from Redis on demand — is your first taste of "the hardest problem in caching." Phase 4 (System Design) goes deep on this.

Request paths, visually



TRY IT YOURSELF

Run `uvicorn main:app --reload` from `simple_cache_server/`, then hit `/user/1` twice with curl or the `/docs` page and watch the second response return almost instantly.

Practice Checklist

Do these yourself in the notebook or your own script before Phase 3. Tick as you go.

- [ ] **Request counter** — extend `get_user_cached` to track how many times each `user_id` is requested using `INCR` on `stats:requests:user:{id}`.
- [ ] **Trending endpoint** — build `/trending` using a **Sorted Set** to hold the 5 most-requested users (score = request count from above).
- [ ] **Session hash** — add a `session:{token}` **Hash** with a 30-minute `EXPIRE`, plus a `/whoami` route that reads it back.
- [ ] **Break it on purpose** — set a 2-second TTL, call the endpoint, sleep 3 seconds, call again — confirm the logs show `CACHE MISS` the second time.
- [ ] **Stretch (Phase 3 preview)** — build a naive rate limiter with a **List**: push a timestamp per request to `ratelimit:{user_id}`, reject if `LLEN` exceeds 10 within the last 60 seconds.

STRING ✓ HASH ✓ LIST ✓ SET ✓ SORTED SET ✓

> Struggling with any exercise? Re-read that data structure's page here first, then re-run the matching cells in the notebook — the notebook's output already shows you the expected shape of the answer.

Your notes & answers

What's Next — Phase 3: SWE Redis

Same five tools. Bigger problems.

Nothing in Phase 3 introduces a brand-new primitive for its main topics — it applies the exact structures you just practiced to real production problems:

Phase 3 topic	Built from
Session Management	Hash + EXPIRE — you already shaped this in the practice checklist
Rate Limiting	INCR + EXPIRE, or Sorted Set with timestamps
Message Queues	List — you already built the core of this on page 6
Pub/Sub	A new primitive — publishers broadcast, subscribers listen

Before you move on, be honest with yourself: can you explain, out loud, without notes, why Hash beats String for a session object, and why RPUSH+LPOP gives you a fair queue? If yes — you're genuinely ready for Phase 3, not just "done reading" it.

Redis Zero to Hero · Phase 2 complete · On to Phase 3



Phase 2 — Redis Fundamentals

String · Hash · List · Set · Sorted Set · Project 1 shipped

Caching Concept ✓ → Redis Fundamentals ✓ → SWE Use Cases →
System Design → AI/ML Use Cases → Agentic AI Use Cases → Production